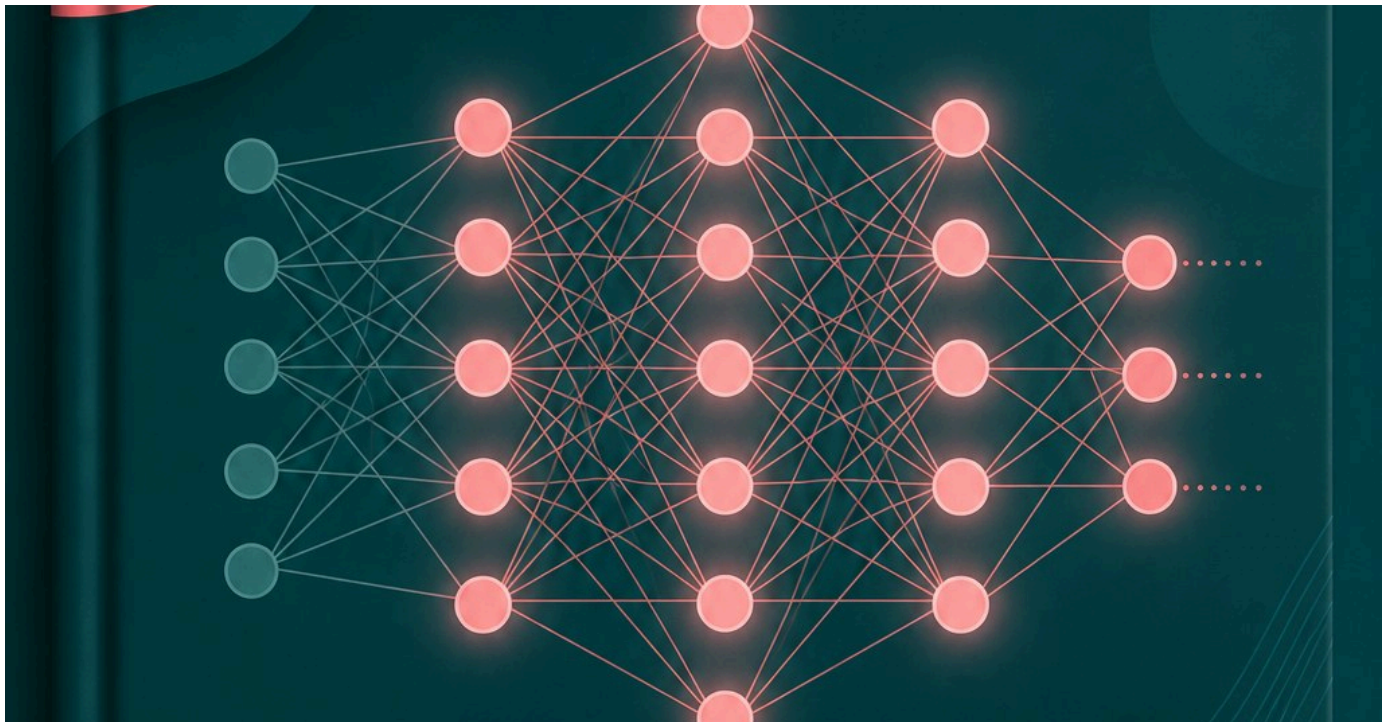




FORMATION IA - DEEP LEARNING

Deep Learning

Neurones, CNN, transformers et limites :
comprendre le deep learning simplement.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - Salut, c'est quoi le deep learning ?

- [Ce que ce livre couvre](#)
- [Deep learning vs ML classique](#)
- [Fil rouge](#)
- [Erreur classique](#)
- [A toi](#)
- [Pourquoi "representations"](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

2. Chapitre 2 - Un neurone : une petite decision ponderee

- [Pourquoi des poids ?](#)
- [Limite d'un seul neurone](#)
- [Lien humain \(et ses limites\)](#)
- [Erreur classique](#)
- [A toi](#)
- [Parametres et capacite](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

3. Chapitre 3 - Couches : empiler des transformations

- [Profondeur](#)
- [Representations](#)
- [Fully connected vs specialise](#)
- [Erreur classique](#)
- [A toi](#)
- [Largeur vs profondeur](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

4. Chapitre 4 - Activations : introduire du non-lineaire

- [Intuitions courantes](#)
- [En sortie](#)
- [Erreur classique](#)
- [A toi](#)

- [Softmax en sortie \(intuition\)](#)
 - [Developpement : ce que le deep learning change vraiment](#)
 - [Intuition des representations](#)
 - [Experimentation sobre](#)
 - [Ethique et impact](#)
 - [Pour aller plus loin sans te perdre](#)
5. [Chapitre 5 - Backpropagation : l'idee simple de l'apprentissage](#)
- [Descente de gradient \(intuition\)](#)
 - [Ce que tu retiens sans maths](#)
 - [Epochs, batches](#)
 - [Erreur classique](#)
 - [A toi](#)
 - [Optimizers \(aperçu\)](#)
 - [Developpement : ce que le deep learning change vraiment](#)
 - [Intuition des representations](#)
 - [Experimentation sobre](#)
 - [Ethique et impact](#)
 - [Pour aller plus loin sans te perdre](#)
6. [Chapitre 6 - CNN : voir par filtres locaux](#)
- [Pourquoi ca marche sur l'image](#)
 - [Usages](#)
 - [Erreur classique](#)
 - [A toi](#)
 - [Data augmentation utile](#)
 - [Developpement : ce que le deep learning change vraiment](#)
 - [Intuition des representations](#)
 - [Experimentation sobre](#)
 - [Ethique et impact](#)
 - [Pour aller plus loin sans te perdre](#)
7. [Chapitre 7 - RNN : sequences dans le temps \(overview\)](#)
- [Limites historiques](#)
 - [Ou les situer en 2026](#)
 - [Erreur classique](#)
 - [A toi](#)
 - [Series temporelles](#)
 - [Developpement : ce que le deep learning change vraiment](#)
 - [Intuition des representations](#)
 - [Experimentation sobre](#)
 - [Ethique et impact](#)
 - [Pour aller plus loin sans te perdre](#)
8. [Chapitre 8 - Transformers : l'attention \(overview\)](#)
- [Attention en image mentale](#)
 - [Encodeurs, decodeurs, seq2seq](#)

- [Lien pratique](#)
- [Erreur classique](#)
- [A toi](#)
- [Contexte et complexite](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

9. [Chapitre 9 - Overfitting en deep learning](#)

- [Remedes courants](#)
- [Generalisation dans le vrai monde](#)
- [Erreur classique](#)
- [A toi](#)
- [Courbes a surveiller](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

10. [Chapitre 10 - GPU : pourquoi ca va plus vite \(idee\)](#)

- [CPU vs GPU \(intuition\)](#)
- [Ce que ca change pour toi](#)
- [Limites](#)
- [Erreur classique](#)
- [A toi](#)
- [Inference vs training](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

11. [Chapitre 11 - Transfer learning : reutiliser un cerveau deja forme](#)

- [Pourquoi c'est le geste 2026](#)
- [Strategies](#)
- [Limites](#)
- [Erreur classique](#)
- [A toi](#)
- [Domaine shift](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

12. Chapitre 12 - Lien avec les LLM : du neurone au chat

- Ce que tu reconnais maintenant
- Entraîner vs utiliser
- Ethique et cout
- Erreur classique
- A toi
- Alignement (idée)
- Developpement : ce que le deep learning change vraiment
- Intuition des representations
- Experimentation sobre
- Ethique et impact
- Pour aller plus loin sans te perdre

13. Chapitre 13 - A retenir

- Boucle DanielCraft DL
- A toi
- Pont vers la pratique
- Developpement : ce que le deep learning change vraiment
- Intuition des representations
- Experimentation sobre
- Ethique et impact
- Pour aller plus loin sans te perdre

14. Chapitre 14 - Atelier : intuition neurone et couches

- Exercices
- Livrable
- Partage
- Developpement : ce que le deep learning change vraiment
- Intuition des representations
- Experimentation sobre
- Ethique et impact
- Pour aller plus loin sans te perdre

15. Chapitre 15 - Atelier : CNN et transfer learning (plan de projet)

- Etapes
- Livrable
- Conseil
- Go / no-go
- Developpement : ce que le deep learning change vraiment
- Intuition des representations
- Experimentation sobre
- Ethique et impact
- Pour aller plus loin sans te perdre

16. Chapitre 16 - Atelier : transformer et usage LLM

- Etapes
- Livrable

- [Budget tokens](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

17. [Chapitre 17 - Choisir une architecture \(carte de decision\)](#)

- [Carte simple](#)
- [Questions de decision](#)
- [Erreur classique](#)
- [A toi](#)
- [Anti-patterns](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

18. [Chapitre 18 - Limites du deep learning](#)

- [Ce qu'il ne remplace pas](#)
- [Robustesse](#)
- [Environnement et cout](#)
- [A toi](#)
- [Abstention](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

19. [Chapitre 19 - Bonnes pratiques deep learning](#)

- [Avant](#)
- [Pendant](#)
- [Apres](#)
- [Usage LLM couple](#)
- [A toi](#)
- [Reproductibilite minimale](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

20. [Quiz final](#)

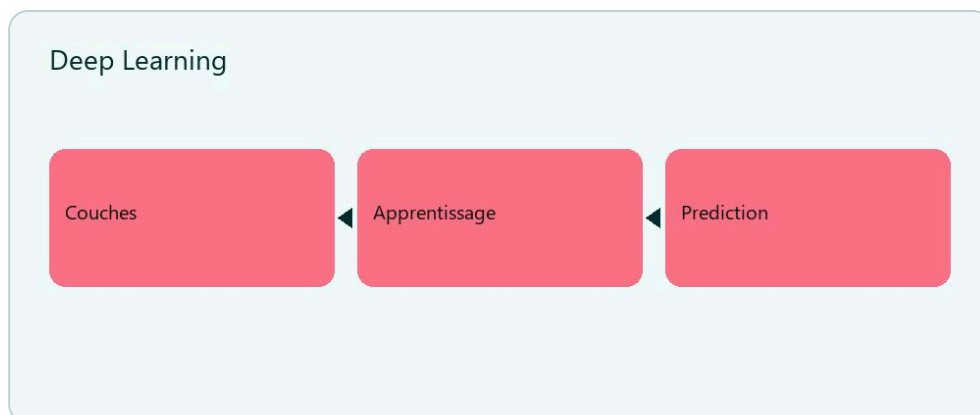
- [Corriges](#)
- [Bonus oral](#)

- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

21. [Chapitre 21 - Bravo](#)

- [La suite](#)
- [Suite possible](#)
- [Developpement : ce que le deep learning change vraiment](#)
- [Intuition des representations](#)
- [Experimentation sobre](#)
- [Ethique et impact](#)
- [Pour aller plus loin sans te perdre](#)

Chapitre 1 - Salut, c'est quoi le deep learning ?



Deep learning : des reseaux a plusieurs couches.

Le deep learning, c'est une famille de machine learning qui utilise des reseaux de neurones artificiels avec plusieurs couches. "Deep" veut dire profond : beaucoup d'etages de transformation entre l'entree et la sortie. Ces modeles ont casse des plafonds sur l'image, la voix, puis le langage. Les LLM dont on parle tous les jours sont, sous le capot, des architectures de deep learning specialisees (souvent des transformers).

Chez DanielCraft, on ne commence pas par les equations. On commence par l'intuition : empiler des filtres qui apprennent a reconnaitre des motifs de plus en plus abstraites - bords, formes, objets, ou pour le texte : fragments, structures, intentions probables.

Ce que ce livre couvre

Neurone, couches, activations, idee simple de la backprop, CNN, RNN, transformers (overview), overfitting en deep learning, idee des GPU, transfer learning, lien avec les LLM, ateliers, limites, bonnes pratiques, quiz. Tu sortiras capable d'expliquer a un ami comment ca marche "en gros", et de ne pas te faire impressionner par un jargon vide.

Deep learning vs ML classique

Sur un petit tableau de 10 colonnes numeriques, un modele scikit-learn simple gagne souvent. Sur une image brute (des milliers de pixels) ou du texte long, le deep learning brille parce qu'il apprend aussi des representations, pas seulement une decision sur des features deja ciselees a la main. En 2026, les deux coexistent. Choisir, c'est matchier le probleme.

Fil rouge

Ines developpe une appli qui reconnait des pieces detachees sur photo, puis veut comprendre comment les assistants texte fonctionnent. Elle n'a pas besoin de reinventer un laboratoire. Elle a besoin d'une carte mentale solide pour dialoguer avec des outils, des prestataires, et plus tard du code.

Erreur classique

Croire que deep learning = toujours mieux. Ou croire qu'il faut un cluster de GPU pour comprendre. Comprendre d'abord ; scaler ensuite.

A toi

Ecris un probleme "image ou texte" ou le deep learning semble pertinent, et un probleme "petit tableau" ou un modele simple suffit. Garde la distinction.

Pourquoi "representations"

En ML classique, tu ciselais souvent les features a la main. En deep learning, les couches apprennent aussi des features internes. Sur une image, tu ne codes pas "bord vertical" a la main ; une convolution peut l'apprendre. Sur du texte, tu n'inventes pas toutes les associations ; l'attention les pondere. Cette automatiser a un prix : opacite, besoin de donnees/calcul, risque d'overfitting. Le jeu n'en vaut la chandelle que si le probleme le demande.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus chers.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entraînement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

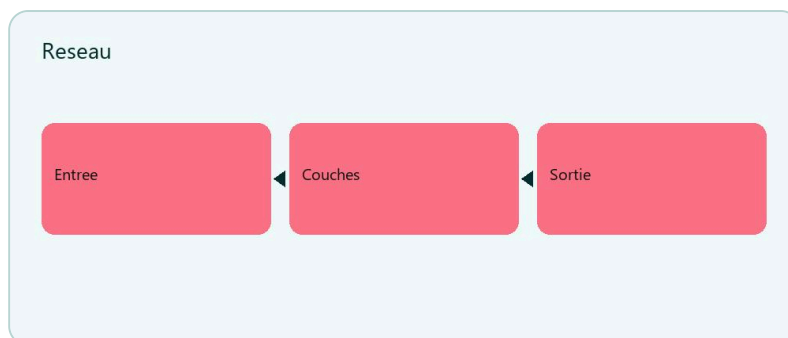
Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatiser de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette

discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 2 - Un neurone : une petite decision ponderee



Neurone artificiel : entrees, poids, sortie.

Un neurone artificiel, en version poche, prend des entrees numeriques, les multiplie par des poids, ajoute un biais, puis passe le resultat dans une fonction d'activation. Image : une recette qui melange des ingredients avec des dosages (poids), puis decide si le signal continue.

Pourquoi des poids ?

Les poids s'ajustent pendant l'apprentissage. Si telle entree est importante pour la tache, son poids grossit (en valeur absolue). Si elle est inutile, elle s'attenuue. Tu n'ecris pas la regle a la main ; tu laisses les exemples pousser les dosages.

Limite d'un seul neurone

Un neurone seul est faible : il trace des frontieres simples. La puissance vient des reseaux : beaucoup de neurones organises en couches, qui composent des decisions. Comme une equipe : chacun voit un aspect ; ensemble ils resolvent un motif complexe.

Lien humain (et ses limites)

Le nom "neurone" s'inspire du cerveau, mais ce n'est pas une copie fidele. C'est une metaphore ingenierie utile. N'en deduis pas qu'un reseau "pense" ou "sent". Il calcule une fonction tres flexible ajustee sur des donnees.

Erreur classique

Imaginer un neurone unique qui "contient" le concept chat. Les concepts emergent souvent de patterns distribues dans beaucoup d'unites. Autre piege : fixer les poids a la main "logiquement" - l'apprentissage le fait mieux sur des taches complexes, a condition d'avoir des donnees.

A toi

Invente 3 entrees pour predire "prendre un parapluie" (meteo, vent, distance). Imagine des poids. Ou ton intuition est-elle deja un mini-neurone ?

Parametres et capacite

Chaque poids est un parametre. Plus tu en as, plus le reseau peut coller a des motifs complexes - et au bruit. La capacite n'est pas un trophée. C'est un budget a depenser avec des donnees et de la regularisation. Un neurone unique a peu de capacite ; un LLM en a une immense.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handcraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 3 - Couches : empiler des transformations

Une couche, c'est un groupe de neurones qui reçoivent les sorties de la couche précédente. Couche d'entrée : tes données (pixels, tokens embeds, mesures). Couches cachées : transformations internes. Couche de sortie : prédiction (classes, nombres, tokens suivants...).

Profondeur

Plus de couches peuvent représenter des fonctions plus riches - mais coûtent plus cher à entraîner, et overfitent plus facilement si les données manquent. "Deep" n'est pas un concours de profondeur. C'est un compromis signal / données / calcul / régularisation.

Représentations

Les premières couches d'un CNN tendent à détecter des motifs locaux simples ; plus loin, des formes plus abstraites. Dans le texte, des couches successives mélangent le contexte. Tu n'as pas à visualiser chaque neurone. Retiens : le réseau apprend aussi comment représenter l'entrée, pas seulement la dernière décision.

Fully connected vs spécialise

Une couche dense relie tout à tout (coûteux sur de grandes entrées). Les CNN partagent des filtres locaux sur l'image. Les transformers utilisent des mécanismes d'attention pour relier des positions du séquence. L'architecture encode un a priori : localité pour l'image, dépendances longues pour le langage, etc.

Erreur classique

Ajouter des couches parce que "deep = moderne" sans mesurer. Ou ignorer la taille d'entrée (aplatir une image 4K dans une couche dense naïve : explosion de paramètres).

A toi

Dessine un réseau à 3 étages pour classer spam/texte court : entrée, cachée, sortie. Que sort la dernière couche (2 scores ? 1 probabilité ?).

Largeur vs profondeur

Elargir une couche (plus de neurones) ou approfondir (plus de couches) ne produit pas le même effet. Trop large et trop profond sans données : overfitting et coût. Les architectures modernes jouent sur des blocs répétés, des connexions résiduelles, des normes - détails que tu rencontreras plus tard. Ici, retiens le levier : forme du réseau = hypothèse sur la structure du problème.

Développement : ce que le deep learning change vraiment

Le deep learning a déplacé le curseur : des tâches autrefois impossibles sans features handcraftées deviennent abordables si tu as des données et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnête, métriques alignées, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systèmes sont plus opaques et plus coûteux.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entraînement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 4 - Activations : introduire du non-lineaire

Si tu n'empilais que des additions et multiplications lineaires, plusieurs couches densent s'effondreraient en une seule transformation lineaire. Les fonctions d'activation cassent cette linearite. Elles permettent au reseau de tordre l'espace et d'approximer des motifs complexes.

Intuitions courantes

ReLU : passe les valeurs positives, coupe les negatives (simple, efficace souvent). Sigmoid / softmax : ecrasent vers des probabilites (souvent en sortie de classification). Il en existe d'autres ; tu n'as pas a les collectionner. Comprends le role : sans non-linearite, pas de profondeur utile.

En sortie

La derniere activation depend de la tache : nombre libre (regression), probabilite (classification binaire), distribution sur classes (softmax), ou distribution sur vocabulaire (LLM). Choisir une mauvaise sortie, c'est comme demander a un thermometre d'afficher une couleur sans legende.

Erreur classique

Utiliser une activation qui sature trop et "tue" le gradient (apprentissage qui n'avance plus) - d'où la popularite de ReLU et variantes dans beaucoup de couches cachees. Autre piege : confondre activation et fonction de perte (loss) : l'une transforme des signaux ; l'autre mesure l'erreur pour apprendre.

A toi

Explique a un ami, en cinq lignes, pourquoi on active non lineairement entre les couches.

Softmax en sortie (intuition)

Pour plusieurs classes, on obtient des scores bruts puis un softmax les transforme en nombres positifs qui somment a 1 : une distribution. Utile pour choisir la classe ou pour lire une incertitude relative. Ce n'est pas une garantie de calibration parfaite, mais c'est le langage courant des classifieurs modernes.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handcraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers

sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisations de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 5 - Backpropagation : l'idée simple de l'apprentissage

La backpropagation (retour d'erreur), c'est la méthode pour ajuster les poids après avoir vu une erreur. Version histoire : le réseau fait une prédiction (avant). On compare à la cible avec une fonction de perte. Puis on propage en arrière de petites indications : "toi, poids, monte un peu ; toi, descends". On répète sur beaucoup d'exemples (souvent par petits lots, mini-batches).

Descente de gradient (intuition)

Imagine être dans le brouillard sur une colline et vouloir descendre : tu sens la pente sous tes pieds et tu fais un pas dans la direction qui descend. Le gradient est cette pente dans l'espace des poids. Le taux d'apprentissage (learning rate) est la taille du pas. Trop grand : tu sautes et tu diverges. Trop petit : tu rampe éternelle.

Ce que tu retiens sans maths

Apprendre = réduire une erreur mesurée en ajustant des millions (parfois milliards) de poids via des passes avant/arrière. Les frameworks (PyTorch, TensorFlow...) calculent les gradients pour toi. Ton job : données, architecture raisonnable, loss adaptée, monitoring, régularisation.

Epochs, batches

Une epoch : un passage sur l'ensemble d'entraînement. Un batch : un sous-groupe d'exemples pour une mise à jour. Ces détails changent stabilité et vitesse. En pratique, tu observes la courbe de loss train/validation.

Erreur classique

Croire que backprop "comprend" le sens. Elle optimise un critère. Si le critère est mal choisi, elle optimise mal. Autre piège : juger un entraînement seulement sur la loss train.

A toi

Raconte la boucle en 6 étapes orales : batch, avant, loss, arrière, maj des poids, répéter. Si tu peux sans notes, c'est gagné.

Optimizers (aperçu)

SGD, Adam et cousins : des façons d'utiliser les gradients pour mettre à jour les poids, avec des astuces de moment et d'adaptation du pas. Tu n'as pas à les réinventer. Sache qu'ils existent, qu'ils ont des réglages, et que le learning rate reste souvent le levier le plus sensible.

Développement : ce que le deep learning change vraiment

Le deep learning a déplacé le curseur : des tâches autrefois impossibles sans features handcraftées deviennent abordables si tu as des données et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnête, métriques alignées, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systèmes sont plus opaques et plus coûteux.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entraînement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

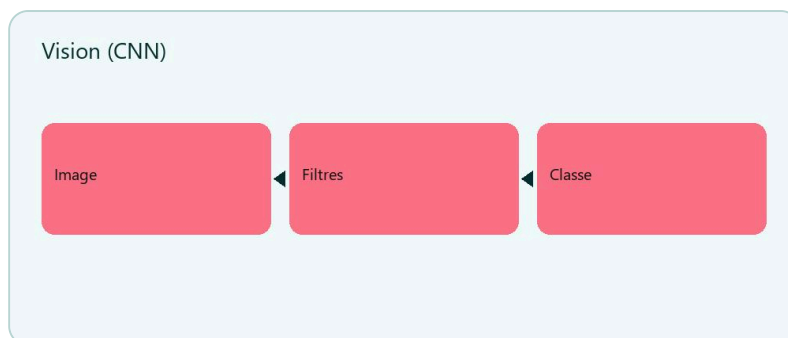
Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 6 - CNN : voir par filtres locaux



CNN : voir des motifs dans les images.

CNN signifie Convolutional Neural Network : reseau a convolutions. Idee : au lieu de relier chaque pixel a chaque neurone (trop cher, trop de parametres), on fait glisser de petits filtres qui detectent des motifs locaux (bords, textures), partages sur toute l'image. Puis on empile : motifs plus riches, cartes de features, souvent reduction spatiale, puis decision.

Pourquoi ca marche sur l'image

Un chat peut etre a gauche ou a droite : un filtre de "oreille" utile partout grace au partage de poids. La localite (un pixel depend surtout de son voisinage) est un a priori puissant. Les CNN ont domine la vision pendant des annees ; d'autres architectures existent aussi aujourd'hui, mais l'intuition convolution reste essentielle.

Usages

Classification d'images, detection d'objets, segmentation, controle qualite sur chaine, lecture de documents scannes (avec d'autres briques). Ines peut entrainer ou, plus souvent, reutiliser un modele preentraîne (transfer learning) sur ses pieces detachees.

Erreur classique

Entraîner un CNN profond from scratch avec 200 photos. Tu overfitteras. Prefere transfer learning + data augmentation (rotation, crop...) avec prudence. Autre piege : juger sur des photos studio et deployer sur des photos floues de telephone.

A toi

Decris 2 filtres mentaux utiles pour ton probleme vision (ex. "contour metallique", "trou de vis"). Pourquoi la localite aide ?

Data augmentation utile

Rotation legere, crop, flip si pertinent, variation de luminosite - pour apprendre l'invariance. N'augmente pas d'une facon qui change le label (un flip peut casser un symbole directionnel). Toujours se demander : cette transformation existe-t-elle dans le vrai monde de deployment ?

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handcraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 7 - RNN : sequences dans le temps (overview)

Les RNN (Recurrent Neural Networks) ont été conçus pour les séquences : texte, audio, séries temporelles. Idée : traiter un élément à la fois en maintenant un état mémoire qui résume le passé. Chaque nouveau mot met à jour la mémoire, puis on prédit.

Limites historiques

Sur les longues séquences, les RNN simples oublient ou deviennent instables (gradients qui disparaissent ou explosent). Des variantes (LSTM, GRU) ont amélioré la mémoire. Puis les transformers ont souvent pris le dessus sur le langage à grande échelle, grâce à l'attention parallèle. Pourquoi en parler encore ? Parce que l'intuition "état qui voyage dans le temps" reste pédagogique, et parce que certaines séries temporelles industrielles utilisent encore des approches récurrentes ou hybrides.

Où les situer en 2026

Pour le langage génératif massif : transformers. Pour comprendre l'histoire du NLP et certaines tâches séquentielles légères : utile. Pour l'audio et la parole, des architectures mixtes existent. L'important : savoir que "séquence" demande un traitement du contexte ordonné, pas seulement un sac de mots.

Erreur classique

Choisir un RNN parce qu'un tutoriel de 2017 le dit, alors qu'un modèle préentraîné transformer résoudrait mieux ton cas texte. Autre piège : croire qu'une mémoire récurrente égale une compréhension humaine du récit.

A toi

Donne un exemple de séquence dans ton métier (logs, phrases, mesures). Qu'est-ce qui dépend du passé récent ?

Séries temporelles

Prédire la demande, détecter une anomalie sur un capteur : parfois RNN/LSTM, parfois modèles statistiques, parfois transformers temporels. La leçon transversale : respecter le temps dans le split (pas de futur qui fuit dans le passé), et évaluer sur des périodes réalistes.

Developpement : ce que le deep learning change vraiment

Le deep learning a déplacé le curseur : des tâches autrefois impossibles sans features handcraftées deviennent abordables si tu as des données et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnête, métriques alignées, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systèmes sont plus opaques et plus coûteux.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de données ? préentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modèle se trompe ? Tu passeras pour quelqu'un de sérieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

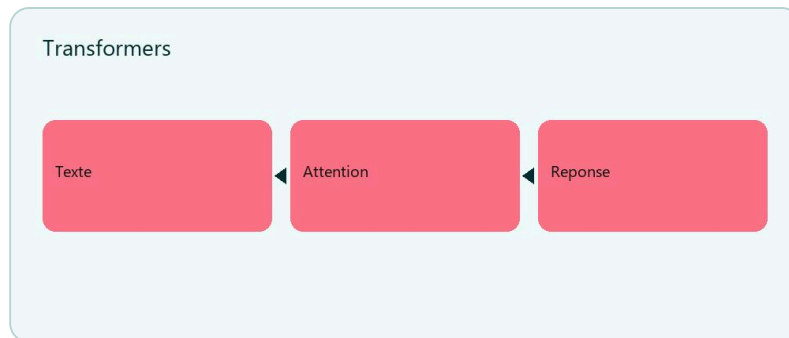
Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 8 - Transformers : l'attention (overview)



Transformers : attention et contexte.

Le transformer est l'architecture dominante du NLP moderne et le coeur de beaucoup de LLM. Idee centrale : l'attention. Chaque position (chaque token) peut regarder les autres positions et ponderer celles qui comptent pour construire une nouvelle representation. Contrairement aux RNN classiques, on peut paralleliser beaucoup mieux sur GPU.

Attention en image mentale

Quand tu lis "le chat sur le tapis, il dort", le mot "il" doit se relier a "chat". L'attention apprend des liens utiles selon la tache. Multi-tetes : plusieurs types de liens en parallele. Empilement de blocs : representations de plus en plus riches. Ajoute des encodings de position (parce que sinon l'ordre serait moins clair).

Encodeurs, decodeurs, seq2seq

Selon les modeles : encodeur seul (classification, embeddings), decodeur seul (generation de texte style GPT), encodeur-decodeur (traduction...). Tu n'as pas a tout memoriser. Retiens : attention + profondeur + plein de donnees + plein de calcul = capacites emergentes de langage.

Lien pratique

Fine-tuning, RAG, prompting : ce sont des facons d'utiliser ces geants sans tout reentrainer. Comprendre le transformer te permet de comprendre pourquoi le contexte a une taille limite, pourquoi les tokens coutent, pourquoi un long document doit parfois etre coupe ou recherches par morceaux.

Erreur classique

Dire "on a mis un transformer" comme garantie de qualite. L'architecture est un moteur ; donnees, alignement, evaluation et usage font le resultat. Autre piege : confondre "attention" technique et "attention" humaine consciente.

A toi

Explique l'attention a un ami avec l'exemple du pronom "il". Puis ajoute : pourquoi un GPU aide a entrainer ca.

Contexte et complexite

L'attention "tous sur tous" coute cher quand la sequence s'allonge (d'ou les limites de contexte et les recherches d'attention efficace). Comprendre ca, c'est comprendre pourquoi on resume, coupe, cherche en RAG, ou paie plus cher pour de longues fenetres.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 9 - Overfitting en deep learning

Les reseaux ont souvent des millions ou des milliards de parametres. Capacite enorme = facilite a coller au train. L'overfitting est donc central. Signes : accuracy train haute, validation basse ; ecart qui se creusent au fil des epochs ; performances magiques sur un petit jeu.

Remedes courants

Plus de donnees reelles. Data augmentation (surtout vision). Dropout (eteindre des neurones au hasard pendant l'entrainement). Weight decay / regularisation. Early stopping (arreter quand la validation n'ameliore plus). Architectures plus petites. Transfer learning plutot que from scratch. Label smoothing parfois. Et toujours : protocole de validation honnete.

Generalisation dans le vrai monde

Meme avec une belle courbe, le deploiement peut casser si les photos changent (eclairage, telephone), si la langue change, si les utilisateurs detournent. Surveillance. Recolte des cas d'echec. Reentraine. Le deep learning n'annule pas la boucle ML : il l'intensifie.

Erreur classique

Augmenter la taille du modele des que ca coince. Parfois il faut mieux de donnees, pas plus de couches. Autre piege : data augmentation absurde qui cree des exemples impossibles et trompe le metier.

A toi

Liste 4 remedes anti-overfitting applicables a ton idee de projet. Lesquels sont realistes cette semaine ?

Courbes a surveiller

Loss train, loss val, metrique metier val. Si train plonge et val remonte : stop / regularise. Si les deux restent hauts : underfitting ou donnees insuffisantes / mal alignees. Apprends a lire ces courbes comme un medecin lit une tension.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraine ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers

sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisations de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 10 - GPU : pourquoi ca va plus vite (idee)

Un GPU (processeur graphique) est tres bon pour faire beaucoup de petites operations similaires en parallele - exactement le genre de calculs des reseaux (grosses multiplications de matrices). D'ou son role central dans l'entrainement et parfois l'inference des modeles lourds.

CPU vs GPU (intuition)

CPU : excellent pour la logique variee, peu de coeurs tres flexibles. GPU : des milliers de petits coeurs pour le throughput. Entrainer un gros CNN ou un transformer sur CPU seul peut etre lent jusqu'a l'absurde. Sur GPU (ou TPU / accelerateurs), ca devient praticable.

Ce que ca change pour toi

Tu n'as pas forcément besoin d'acheter une carte. Cloud, Google Colab, services managés, modeles deja entraines via API : autant de facons d'utiliser la puissance sans la posseder. Comprends juste pourquoi "il me faut un GPU" apparait dans les tutos, et pourquoi l'inference LLM a un cout (calcul = argent + energie).

Limites

Memoire GPU (VRAMs) : un gros batch ou un long contexte peuvent saturer. Optimisations : precision reduite, quantization, distillation, batching. Encore une fois : idees, pas besoin de tout implementer jour 1.

Erreur classique

Croire que sans GPU perso on ne peut rien apprendre. Tu peux comprendre, prototyper petit, utiliser des modeles preentraines, appeler des API. Autre piege : louer du calcul cher sans d'abord valider la question metier sur un sous-ensemble.

A toi

Estime pour ton projet : est-ce que tu as besoin d'entrainer, de fine-tuner, ou seulement d'inferer via un modele existant ? Le besoin GPU change selon la reponse.

Inference vs training

Entrainer est souvent le plus gourmand. Inferer (servir des predictions) peut aussi couter cher a grande echelle (millions de requetes LLM). Optimiser l'inference (quantization, batching, petit modele) est un metier. Pour un debutant : mesure avant d'acheter du fer.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 11 - Transfer learning : reutiliser un cerveau deja forme

Le transfer learning, c'est repartir d'un modele deja entraine sur une tache large (ex. reconnaitre des objets sur des millions d'images, ou un modele de langage sur d'énormes corpus) et l'adapter a ton cas avec moins de donnees. Au lieu d'apprendre tout from scratch, tu reutilises des representations utiles.

Pourquoi c'est le geste 2026

Parce que les donnees labellisees coutent cher, et parce que les modeles fondation existent. Ines telecharge un CNN preentraine, remplace la tete de classification, entraine surtout les dernieres couches sur ses pieces. Sur le texte, on fine-tune legerement, ou on fait du prompting / RAG sans tout retoucher. Le transfer learning est l'esprit ; le prompting est parfois la forme extreme "sans maj des poids".

Strategies

Geler beaucoup de couches, entrainer la tete. Puis parfois deverrouiller plus profond a petit learning rate. Adapter avec peu de parametres (LoRA et cousins dans le monde LLM). Surveiller l'overfitting : meme un modele preentraine peut coller a 100 images.

Limites

Si ton domaine est trop eloigne (images medicales tres specifiques vs photos web), le transfert aide moins ou demande plus de soin. Si tes labels sont sales, tu transfere aussi vers le sale. La licence et le cout des modeles comptent.

Erreur classique

Reentrainer tout a gros learning rate et detruire les representations utiles (catastrophe forgetting). Ou croire que transfer learning dispense de validation.

A toi

Decris un modele fondation que tu reuserais (vision ou texte) et ce que tu adapterais (classes, style, documents).

Domaine shift

Si tes images de pieces sont greasy, floues, mal eclairees, et que le modele preentraine a vu des photos web propres, tu as un ecart de domaine. Le transfert aide encore souvent, mais prevoyez plus d'exemples cibles, une augmentation realiste, et une evaluation sur le vrai terrain.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraine ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

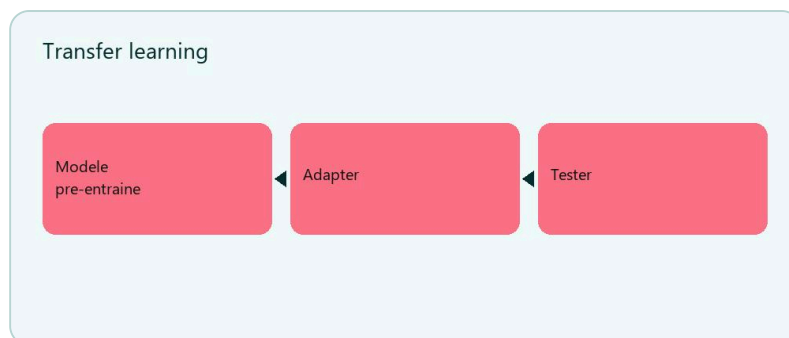
Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 12 - Lien avec les LLM : du neurone au chat



Des reseaux profonds aux LLM.

Un LLM est un reseau de deep learning (souvent transformer decodeur) entraine a predire le prochain token sur d'énormes corpus, puis souvent aligne pour mieux suivre des instructions et respecter des politiques. Quand tu chates, tu ne "parles pas a une conscience". Tu conditionnes une machine a suites de tokens.

Ce que tu reconnais maintenant

Tokens et contexte : la fenetre limite ce que l'attention peut croiser d'un coup. Temperature : reglage sur l'aleas des choix de tokens. Poids et couches : milliards de parametres ajustes par descente de gradient a l'echelle industrielle. Hallucinations : prediction plausible hors verite. Multimodal : on branche vision/audio sur des representations communes ou des modules couples. RAG : on injecte des extraits dans le contexte plutot que de tout memoriser dans les poids. Agents : on boucle le LLM avec des outils.

Entraîner vs utiliser

Entraîner un LLM fondation : hors de portee de la plupart des individus (donnees, GPU, argent, energie). Utiliser : API, produit grand public, modele open poids local selon machine. Fine-tuning léger : parfois accessible. Prompting et RAG : levier numero un. Comprendre le deep learning te protege des fantasies ("il suffit d'uploader mon PDF pour qu'il sache tout pour toujours") et t'oriente vers les bons leviers.

Ethique et cout

Derriere le chat fluide : calcul, eau/energie selon les infrastructures, travail d'annotation et de moderation, risques de biais. Le chapitre n'est pas un proces. C'est une invitation a utiliser avec mesure, surtout en entreprise.

Erreur classique

Croire qu'un LLM rend inutile le ML classique sur un CSV metier. Souvent non. Autre piege : confondre "fine-tune" marketing et vrai changement de poids evalue proprement.

A toi

Relie en 8 fleches : donnees -> transformer -> tokens -> prompt -> reponse -> verification. Ou places-tu RAG et temperature ?

Alignement (idee)

Apres l'entrainement "predire le prochain token", beaucoup de LLM passent par des etapes pour mieux suivre les instructions et reduire certains comportements dangereux. Ca n'efface pas les hallucinations. Ca change le temperament du produit que tu utilises. D'ou des differences de style entre outils.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 13 - A retenir

Deep learning = reseaux a plusieurs couches. Neurone = melange pondere + activation. Couches = transformations empilees. Activation = non-linearite. Backprop = ajuster les poids via l'erreur. CNN = filtres locaux pour l'image. RNN = memoire sequentielle (overview historique utile). Transformers = attention, coeur des LLM. Overfitting = capacite vs donnees. GPU = parallelisme pour matrices. Transfer learning = reutiliser un modele fondation. LLM = deep learning langage + alignement + usage (prompt/RAG/agents).

Boucle DanielCraft DL

Tache claire -> assez de donnees / transfert -> architecture adaptee -> loss et metriques -> validation -> regularisation -> deploiement mesure -> surveillance. Prefere reutiliser avant de reentrainer le monde.

A toi

Carte mentale une page, TES mots, un exemple vision et un exemple texte.

Pont vers la pratique

Si tu utilises deja ChatGPT : tu manipules un transformer aligne. Si tu classes des images : tu toucheras CNN/transfer. Si tu as un CSV : reviens au livre ML. La bonne stack est celle qui matche le probleme.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraine ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 14 - Atelier : intuition neurone et couches

Objectif : solidifier l'image mentale sans code obligatoire. Duree : 30 minutes.

Exercices

1) Dessine un neurone a 3 entrees, poids, biais, activation. Invente des nombres et calcule a la main une sortie ReLU simple. 2) Empile 2 couches denses sur papier (tailles 3 -> 4 -> 1). Explique oralement le passage avant. 3) Ecris la boucle d'apprentissage en 6 etapes. 4) Identifie pour ton projet : entree brute, representation utile, sortie. 5) Option code : un mini reseau toy dans un notebook (meme 20 lignes) pour voir la loss baisser sur une fonction simple.

Livable

Une feuille dessinee + 10 lignes d'explication que tu pourrais envoyer a un collegue non tech.

Partage

Explique ton dessin a quelqu'un. Si cette personne peut reformuler la backprop en une phrase, ton atelier a reussi. L'enseignement est le meilleur test de comprehension.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entraînement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 15 - Atelier : CNN et transfer learning (plan de projet)

Objectif : écrire un plan de projet vision realiste. Duree : 40 minutes.

Etapas

1) Définir 3 a 10 classes d'images. 2) Estimer combien de photos par classe tu peux collecter. 3) Decrire les conditions reelles (eclairage, flou, telephone). 4) Choisir un modele preentaine a adapter. 5) Plan d'augmentation de donnees (liste). 6) Split train/val/test. 7) Metriques (accuracy + erreurs couteuses). 8) Plan anti-overfitting. 9) Critere go/no-go avant deploiement. 10) Idee GPU : local, cloud, ou seulement inference API.

Livrable

Document "plan CNN" de 1 a 2 pages. Pas besoin d'avoir entraine pour valider le plan.

Conseil

Si tu as moins de 50 images au total, reste sur proof of concept et transfer learning agressif, ou reconsidere la faisabilite.

Go / no-go

Ecris a l'avance : "si la metrique X sur test terrain $< Y$, on n'automatise pas, on garde un humain dans la boucle". Signe-le. Tu evites le deploiement par enthousiasme.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentaine ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 16 - Atelier : transformer et usage LLM

Objectif : relier architecture et gestes d'usage. Duree : 35 minutes.

Etapas

1) Explique l'attention avec ton propre exemple de phrase. 2) Mesure mentalement une limite de contexte : que coupes-tu dans un long PDF ? 3) Concevoir un mini-RAG : 5 documents, question, extrait a coller, consigne anti-invention. 4) Deux temperatures mentales sur la meme tache (strict vs creatif) et grille d'evaluation. 5) Decrire un fine-tuning que tu NE feras pas maintenant, et pourquoi prompting/RAG suffisent. 6) Lister 3 risques (donnees, hallucination, cout tokens).

Livrable

Fiche "du transformer a mon usage" avec schema attention + protocole RAG manuel + regles de cout.

Budget tokens

Estime le cout mensuel d'un usage : questions/jour *tokens moyens* prix. Ajoute le cout de tes erreurs (temps de verification). Decide si un modele plus petit suffit pour 80 % des cas.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entraînement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 17 - Choisir une architecture (carte de decision)

Tu n'as pas à inventer une architecture nouvelle. Tu as à choisir une famille adaptée.

Carte simple

Petit tableau numérique : ML classique d'abord. Images : CNN ou vision transformer préentraîné + transfert. Texte génération / chat : LLM existant + prompt/RAG. Séries temporelles : modèles spécialisés ou approches mixtes ; ne force pas un LLM partout. Audio : modèles parole/audio préentraînés. Multi-tâches complexes : parfois pipelines (vision puis règles puis LLM).

Questions de decision

Combien de données labellisées ? Quel budget calcul ? Quelle latence acceptable ? Quel besoin d'interprétabilité ? Quel risque d'erreur ? Existe-t-il un modèle fondation proche ? Peut-on résoudre sans deep learning ?

Erreur classique

Partir d'une architecture parce qu'elle est dans un papier à la mode. Pars du problème. Chez DanielCraft, le prestige du papier ne paie pas les faux positifs clients.

A toi

Remplis la carte pour 2 problèmes de ton monde. Architecture choisie + raison + alternative.

Anti-patterns

LLM pour classer 3 catégories sur un CSV de 20 colonnes. CNN from scratch avec 40 images. Agent autonome le jour 1. Transformer "parce que c'est moderne" sur une série temporelle ou un modèle simple suffit. Note tes anti-patterns personnels.

Developpement : ce que le deep learning change vraiment

Le deep learning a déplacé le curseur : des tâches autrefois impossibles sans features handcraftées deviennent abordables si tu as des données et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnête, métriques alignées, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systèmes sont plus opaques et plus coûteux.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de données ? préentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modèle se trompe ? Tu passeras pour quelqu'un de sérieux. C'est voulu.

Intuition des representations

Une bonne représentation rend le problème plus simple pour la couche suivante. Au début du réseau, l'entrée est brute (pixels, tokens). Au milieu, des motifs. À la fin, une décision. Transfer learning = réutiliser un milieu déjà riche. RAG cote LLM = injecter des faits dans le contexte plutôt que de tout stocker dans les poids. Prompting = conditionner la représentation de sortie sans maj de poids. Ces leviers sont différents, mais ils parlent le même langage : influencer ce que le système "voit" avant de décider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 18 - Limites du deep learning

Le deep learning est puissant et borne. Il demande des donnees, du calcul, de l'ingenierie. Il peut etre opaque. Il peut etre brittle hors distribution. Il peut encoder des biais. Il peut couter cher en inference a grande echelle. Il peut donner une illusion de comprehension.

Ce qu'il ne remplace pas

La definition du probleme. La qualite des labels. La decision ethique. La responsabilite legale. La connaissance terrain. Les modeles simples quand ils suffisent. Les processus humains de verification.

Robustesse

Change l'eclairage, le accent, le format de document, la saison : les performances chutent parfois. Les adversaires peuvent tromper certains modeles avec des perturbations. En production, prevoyez des replis (regles, humain, abstention).

Environnement et cout

Entraîner et servir de gros modeles a un impact energie / argent. Utilise le plus petit modele qui fait le job. Prefere reutiliser. Mesure avant d'empiler.

A toi

Ecris 5 limites qui s'appliquent a ton projet. Pour chacune, une mitigation (humain, regle, plus de donnees, modele plus petit...).

Abstention

Parfois la meilleure sortie est "je ne sais pas / humain requis". Concois un seuil d'abstention. En vision comme en langage, l'abstention sauve des catastrophes.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 19 - Bonnes pratiques deep learning

Avant

Valider que le deep learning est necessaire. Chercher un modele preentaine. Estimer donnees et calcul. Definir metriques et risques. Preparer un set de validation propre.

Pendant

Commencer petit. Logger loss et metriques. Early stopping. Regulariser. Inspecter les echecs. Ne pas toucher au test final trop souvent. Documenter hyperparametres et versions.

Apres

Evaluer hors distribution. Monitorer. Prevoir reentrainement. Versionner. Separer experiences jouets et systeme de production. Informer les utilisateurs des limites.

Usage LLM couple

Si ton systeme mele CNN + LLM + regles, teste chaque brique, puis l'ensemble. Un pipeline opaque a trois etages multiplie les hallucinations operationnelles.

A toi

Checklist 12 cases. Coche sur ton prochain essai, meme toy.

Reproductibilite minimale

Graine, version code, version donnees, config, commit, metriques. Sans ca, tu ne sauras pas pourquoi "ca marche plus". Une demi-heure de rangement economise des jours.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentaine ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Quiz final

1. 1. Le "deep" de deep learning renvoie surtout a :

- A) La couleur sombre des GPU
- B) Des reseaux a plusieurs couches
- C) Une voix grave

1. 2. Une activation non lineaire sert a :

- A) Decorar les graphiques
- B) Permettre aux couches empilees d'approximer des motifs complexes
- C) Remplacer les donnees

1. 3. La backprop, en idee :

- A) Ajuste les poids a partir de l'erreur
- B) Efface le disque dur
- C) Genere des images sans modele

1. 4. Un CNN est particulierement utile pour :

- A) Les filtres locaux sur images
- B) Remplacer toute base de donnees
- C) Facturer les API

1. 5. Les transformers reposent surtout sur :

- A) L'attention entre positions / tokens
- B) Une unique droite de regression
- C) Un seul neurone

1. 6. L'overfitting DL se manifeste souvent par :

- A) Train excellent, validation faible
- B) Train faible, validation parfaite toujours
- C) Absence totale d'erreur humaine

1. 7. Un GPU aide surtout parce que :

- A) Il parallelise de gros calculs de matrices
- B) Il ecrit tes prompts
- C) Il anonymise les donnees tout seul

1. 8. Le transfer learning, c'est :

- A) Reutiliser un modele preentraine pour l'adapter
- B) Transferer des fichiers par FTP uniquement
- C) Interdit en 2026

1. 9. Un LLM est :

- A) Sans aucun lien avec le deep learning
- B) Un modele de deep learning (souvent transformer) pour le langage
- C) Un type de cable

1. 10. Avant d'entrainer un gros reseau from scratch, DanielCraft recommande souvent :

- A) De regarder si un modele preentraine / une API / un modele simple suffit
- B) D'acheter dix GPU sans question metier
- C) D'ignorer la validation

Corriges

1-B, 2-B, 3-A, 4-A, 5-A, 6-A, 7-A, 8-A, 9-B, 10-A.

9/10+ : bon socle. Sinon, relis couches, backprop, CNN/transformers, transfer, lien LLM.

Bonus oral

Sans notes, explique a un ami : neurone, couche, backprop, CNN, transformer, transfer learning, lien LLM. Si tu bloques sur un, relis le chapitre, recommence.

Developpement : ce que le deep learning change vraiment

Le deep learning a deplace le curseur : des taches autrefois impossibles sans features handicraftées deviennent abordables si tu as des donnees et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnete, metriques alignees, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systemes sont plus opaques et plus couts.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de donnees ? preentraine ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modele se trompe ? Tu passeras pour quelqu'un de serieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

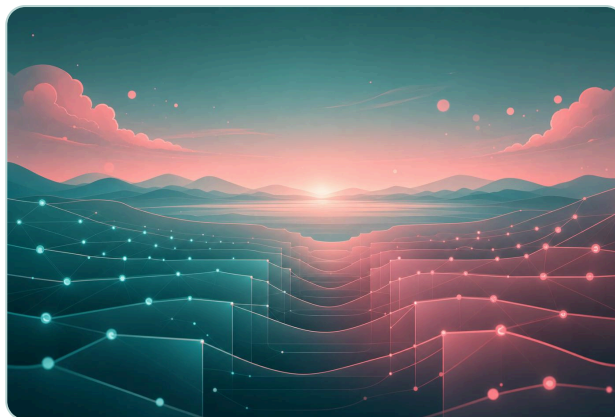
Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Chapitre 21 - Bravo



Bravo. Tu as une carte du deep learning.

Tu as maintenant une carte du deep learning : neurones, couches, activations, apprentissage par retour d'erreur, CNN, RNN en overview, transformers, overfitting, GPU, transfer learning, et le pont vers les LLM que tu utilises peut-être déjà tous les jours.

Chez DanielCraft, on croit à cette clarté : tu n'as pas besoin de prétendre que tu dérivés des gradients sur un tableau avant le café. Tu as besoin de savoir ce qui se passe sous le capot assez pour choisir, questionner, et ne pas acheter de la magie.

La suite

Si tu viens du livre IA générative, tu boucles la boucle usage + mécanisme. Si tu viens du machine learning, tu vois quand passer au deep - et quand rester simple. Ensuite : un petit projet vision ou un usage LLM + RAG propre, évalue, sécurise.

Bravo. Empile des couches de compétence, pas seulement des couches de neurones.

Suite possible

1) Mini classification d'images par transfer learning. 2) Usage LLM + RAG évalué. 3) Relire le livre ML pour les métriques et biais. Tu as le vocabulaire ; il te faut des cicatrices d'expériences - les bonnes.

Developpement : ce que le deep learning change vraiment

Le deep learning a déplacé le curseur : des tâches autrefois impossibles sans features handcraftées deviennent abordables si tu as des données et du calcul. Images, parole, langue. Mais il n'a pas aboli les fondamentaux : split honnête, métriques alignées, biais, monitoring, abstention. Il les a rendus plus urgents, parce que les systèmes sont plus opaques et plus coûteux.

Quand tu entends "on a mis du deep learning", pose les questions de ce livre : combien de données ? préentraîne ou from scratch ? quelle validation ? quel GPU / quel budget ? quel comportement hors distribution ? quel plan si le modèle se trompe ? Tu passeras pour quelqu'un de sérieux. C'est voulu.

Intuition des representations

Une bonne representation rend le probleme plus simple pour la couche suivante. Au debut du reseau, l'entree est brute (pixels, tokens). Au milieu, des motifs. A la fin, une decision. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers sont differents, mais ils parlent le meme langage : influencer ce que le systeme "voit" avant de decider.

Experimentation sobre

Change une chose a la fois. Logge. Compare a une baseline. Arrete-toi quand la validation stagne. Ne confonds pas agitation et progres. Un entrainement qui fait baisser la loss train sans ameliorer la val n'est pas un succes. Un petit modele qui generalise un peu est parfois preferable a un grand modele qui memorise.

Ethique et impact

Derriere les perfs : energie, annotation, risques de surveillance, deepfakes, automatisation de decisions sensibles. Ce livre introductif ne tranche pas tous les debats. Il exige que tu les voies. Utiliser un outil puissant sans regarder ses effets collateraux, ce n'est pas de la neutralite technique. C'est une decision. Assume-la.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel. Relis l'erreur classique. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple. Des que ca passe a l'oral, c'est que c'est entre. Ensuite seulement, passe au chapitre suivant. Cette discipline lente cree une competence rapide sur la duree - le contraire du binge de tutos oublies le lendemain.

Tu as les briques. Maintenant, construis.